Brendan Martel
BXM240013

Project 3 report

Prepossessing steps

Before discussing the cnn and mlp I first need to discuss how I processed the data to make it easier for the models to use. I achieved this using standard normalization bringing all of the numbers between [-1,1]. I achieved this by utilizing the transform option when fetching the data from tourchvision. This allows you to define and apply your own transform to the dataset. I first started by sending the data to a tensor and then using the inbuilt normalize function. I supplied the normalize function with the means and standard deviations of each of the layers allowing the function to properly normalize the data. These means and standard deviations were found in the official pytourch documentation.

In addition to prepossessing the data I also made sure to set a seed for both the torch and cuda to ensure consistency across runs. I used the number 56 for the seed.

Testing and validation splits

For the testing and validation splits I chose a .83, .17 training validation split for MNIST this got me close to my desired 50k 10k split. For the cifar10 I chose a .9, .1 validation split which got me the desired 45k, 5k desired testing and validation split.

Hyper parameter tables with validation results

MNIST Results

Architecture	Learning Rate	Batch Size	Optimizer	Dropout	Validation Acc	Runtime (mins)
MLP (shallow, 1 hidden)	0.01	32	SGD	0	0.919019608	7.803727182
MLP (medium, 3 hidden)	0.001	32	Adam	0.2	0.97745098	3.524678389
MLP (deep, ≥5 hidden)	0.001	64	Adam	0.2	0.978431373	3.467190421
Test accuracy on final MLP model: 0.9743						
CNN (baseline, 2 conv)	0.001	64	Adam	N/A	0.992058824	4.588807702
CNN (enhanced, BN+dropout)	0.001	64	Adam	0.2	0.992058824	4.682617696
CNN (deeper, ≥3 conv)	0.001	64	Adam	0.2	0.99372549	4.797002681
Test accuracy on final CNN model: 0.9895						

CIFAR10 Results

Architecture	Learning Rate	Batch Size	Optimizer	Dropout	Validation Acc	Runtime (mins)
MLP (shallow, 1 hidden)	0.0001	128	Adam	0.5	0.3946	4.468097123
MLP (medium, 3 hidden)	0.0001	128	Adam	0.5	0.529	2.880932315
MLP (deep, ≥5 hidden)	0.0001	128	Adam	0.5	0.5244	2.958568803
Test accuracy on final MLP model: 0.5281						
CNN (baseline, 2 conv)	0.01	32	SGD	N/A	0.72	5.042761584
CNN (enhanced, BN+dropout)	0.01	32	SGD	0	0.707	3.953366351
CNN (deeper, ≥3 conv)	0.001	128	Adam	0.5	0.7502	3.010463083
Test accuracy on final CNN model: 0.7266						

This table includes the results from the top performing runs for each of the different architectures on each of the different datasets. The data this table was created from can be found in the zip under the modelDatasetlogging.csv for your desired set and model.

Hyper parameters tuning

For this project I tuned the hyper parameters from a predefined set of each hyper parameter. The sets are seen below:

```
learningRates = [0.01,0.001,0.0001]
batchSize = [32,64,128]
optomizer = ["SGD", "Adam"]
dropRates = [0,0.2,0.5]
```

When tuning the hyper parameters I performed a random search through the hyper parameters. I first started with the SGD optimizer, I then ran the model with the parameters from the 0 index of the array. I then proceeded to log this increment the index by one and run it until the index had reached 2. I would then switch to Adam and repeat the process.

After receiving this result for the model I would then manually attempt to tune the parameters based on the highest validation accuracy. The only exception to this is when the validation accuracy was 99%. When this happened I did not tune further and instead took the top model. I trained all of my models with 20 epochs and a stopping factor of 10.

Selecting the final model

After tuning the hyper parameters in the above way I then had to pick the final model. This was easy as I had already explored combinations nearby to the highest validation that was achieved using the random search. So it was simply a mater of choosing the model with the parameters that netted the highest validation score. This is why in the table you can find the model that was used for testing by finding the model that had the highest validation accuracy. This method would hopefully net the best result for testing.

Architecture chosen

For both of the architectures Multilayer perceptrons (MLPs) and Convolutional Neural Nets (CNNs) I followed what the assignment asked for very closely.

MLPs

For the MLPs I tested the three architectures suggested by the project. A shallow MLP consisting of only a single hidden layer. A medium MLP consisting of 3 hidden layers, and finally a deep MLP with 5 layers. These three architectures were then applied to the two different sets.

CNNs

For the CNNs I tested the three architectures suggested by the project. A baseline CNN with only 2 layers of convolution, pooling, and no dropout layers. Then an enhanced version which includes all of the last architecture but additionally includes batch normalization and dropout layers. Finally, the Deeper CNN which includes all features of the enhanced CNN but also includes an additional convolutional layer.

Why certain things worked better than others

I think that going into this project it was clear that the CNNs would likely outperform the MLPs. I made this assumption purely off the complexity of the different models and what we had seen from the lecture. It turns out that this gut reaction was in fact correct but not across the board. If we

look at the results for the MNIST data set we can actually see that the difference between the models is marginal only about a percent or two between them.

This is likley because of the single channel nature of the input. Because the images are simply black and white the information within them flattens down to a single value simply and therefore is suited to both a complex model like the CNN and a less complex model like the MLPs that are simply input and output with additional hidden layers. This is also due to the lower amount of inputs potentially discouraging over fitting from the smaller 28x28 as compared to the larger images in the CIFAR10 dataset. You can actually see that the medium performed better in the CIFAR10 rather then the Deeper model and this may have been one potential factor in that. The CNN attempts to refine the image through its different layers and can handle additional channels with ease. Because of this we can see the major differences come out on the CIFAR10 which are color images. Because of the loss of information when flattening down the MLPs perform much worse then the CNNs. Meanwhile the CNNS thrive with the additional information. We can see this through the difference in accuracy.

As for the hyper parameters I noticed a major difference with the overall dominance of the Adam optimizer. From the table it is clear that Adam outperformed SGD on many of the runs. I think that this difference may be due to the differences in the two optimizer. While Adam works adaptability and is less sensitive to hyper parameter tuning it is also prone to converging much faster than SGD. This would make sense as all of these tests were only done over 20 epochs. SGD slows down as it approaches the minima and requires more steps in order to converge. This could be a reason for the major imbalance in appearances in the top performing models.

A difficulty that I ran into

The biggest difficulty for me was just understanding the PyTorch library. While the library is obviously both strong and relevant to modern applications of ml I knew nothing about it coming into the project. Because of that of the ~30 hours spent on this project the majority of those hours were just spent learning what was going on and how to connect all the various dots of the PyTorch library. While I did eventually get it the initial startup was a struggle.

I was able to overcome this challenge with the use of AI and the PyTorch wiki. The wiki provided me with a massive amount of knowledge but was difficult to understand for a complete beginner especially given what we were supposed to apply it too. AI was useful in explaining what everything did in terms that I was already familiar with making the process of learning simpler than just reading bland documentation. If I had to make a recommendation for the future I would suggest providing a template or more targeted information then just the entire wiki.